Aufgabe 2

Auswertung eines Drehgebers durch Polling

Aufgabenbeschreibung

Bei einem rotierenden Gegenstand wird mit Hilfe eines inkrementellen Drehgebers der Drehwinkel ermittelt. Oft besteht ein Drehgeber aus einer Scheibe mit Schlitzen. Wenn sich die Scheibe dreht, wird abwechselnd der Weg für einen Lichtstrahl von einer LED zu einer Photodiode versperrt und wieder frei gegeben. Zur Ermittlung der Drehrichtung wird eine zweite Photodiode ein wenig versetzt angebracht (siehe Abbildung 1).

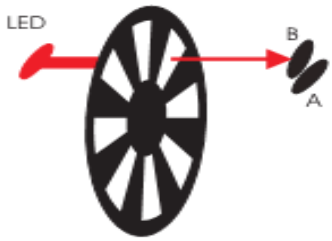

Abb. 1: *Funktionsprinzip Drehgeber mit versetzten Photodio-*
c

Solch eine Anordnung ist im TI-Labor als Lehrmodell mit einer Drehscheibe aufgebaut (siehe. Abbildung 2). Für die einfache Entwicklung auf dem ITS-Board werden die Signale des Drehgebers durch ein Programm auf dem Raspberry Pi Pico nachgebildet/simuliert. Details zum Aufbau des Drehreglers (Datenblatt_Drehregler.pdf) und des Mockups (Rechteckgenerator.zip) finden Sie in TEAMs.

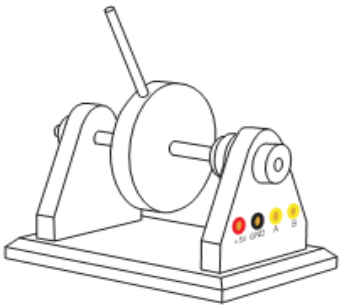

Abb. 2: *Drehgeber im Labor*

Die Werte der Dioden sind auf die Kanäle A und B abgebildet. Wie in Abbildung 3 gezeigt, lassen sich die Impulsverläufe an Kanal A und Kanal B in vier Phasen a, b, c und d einteilen: Bei einer Vorwärtsbewegung werden die Phasen zyklisch in der Reihenfolge a, b, c und d durchlaufen. Bei einer Rückwärtsbewegung werden die Phasen in umgekehrter Reihenfolge durchlaufen. Durch Beobachtung der Phasen lässt sich die Bewegungsrichtung der Scheibe und die Anzahl der durchlaufenen Phasen feststellen. Ist die aktuelle Phase b so ergeben sich bei der nächsten Beobachtung der Signale die Fälle aus Tabellen 1 und 2.

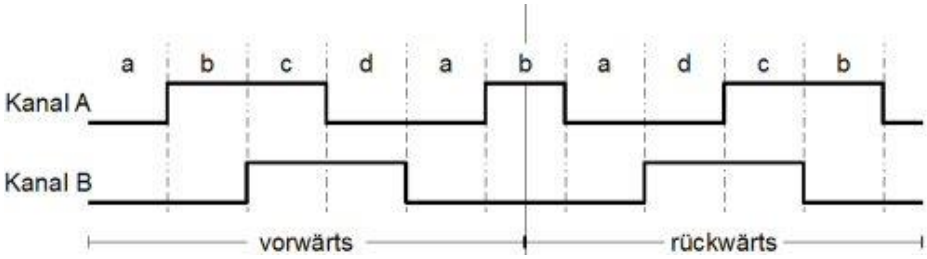

Abb. 3: *Ausgangssignale des Drehgebers in Abhängigkeit des Drehwinkels*

Versuchsaufbau

Der Drehgeber bzw. der Pi Pico (zur Simulation des Drehgebers) wird über die Pins `IN0` (`GPIO PF0`) und `IN1` (`GPIO PF1`) mit den ITS-Board verbunden. Beachten Sie bei der Nutzung des Drehgebermodells im Labor, dass der Drehgeber mit 3,3V über das ITS-Board versorgt wird.

Zuordnung Signale - Phase		
Kanal A	Kanal B	Phase
0	0	a
1	0	b
1	1	c
0	1	d

Tab. 1: *Phasen*

Zuordnung Phasenwechsel - Ereignis	
Phasenwechsel	Ergebnis
Phase b →Phase a	Rückwärtslauf
Phase b →Phase b	Keine Änderung
Phase b →Phase c	Vorwärtslauf
Phase b →Phase d	Fehler

Tab. 2: *Phasenübergänge*

Die zu `IN0` und `IN1` gehörigen LEDs zeigen den Status der Pins / Photodioden an. Bei langsamen Drehen erkennt man an den LEDs die einzelnen Phasen und kann somit schnell überprüfen, ob die Hardware korrekt angeschlossen ist. Mit Blick auf die Qualitätssicherung Ihres Programms ist der Drehgeber ungenau. Bitte nutzen Sie den auf dem Pi Pico implementierten Rechteckgenerator zum Test Ihres Programms. Während der Abnahme der Aufgabe werden Drehgeber und Pi Pico verwendet.

Allgemeine Anforderungen

- Unter Beachtung der Drehrichtung wird die Anzahl der Schritte / Phasenwechsel - nicht der Winkel in Grad - binär codiert an den LEDs `D8` bis `D15` ausgegeben. Sie sind an den GPIOs `PD0` bis `PD7` angeschlossen. Es werden nur die niederwertigsten 8 Bits dieses Vorwärts- Rückwärtszählers angezeigt.
- Auf dem LCD-Display werden der Drehwinkel in Grad und die Drehgeschwindigkeit / Winkelgeschwindigkeit in Grad pro Sekunde ausgegeben. Der Winkel muss exakt angegeben werden, die Winkelgeschwindigkeit darf im Nachkommabereich abweichen. Zur Bestimmung der Winkelgeschwindigkeit benötigen Sie einen Timer zur Zeitmessung (`timer.h` in der ITS_BRD_LIB Bibliothek).
- LED `D23` leuchtet, wenn als letztes eine Drehung in Vorwärtsrichtung erkannt wurde.
- LED `D22` leuchtet, wenn als letztes eine Drehung in Rückwärtsrichtung erkannt wurde.
- Im Fehlerfall leuchtet LED `D21`. Sie wird durch Löschen des Fehlers zurückgesetzt.
- Mit der Taste `S6` wird der Fehlerzustand gelöscht.
- Führen Sie die Winkel- und Winkelgeschwindigkeitsberechnung mit dem Typ `double` aus.
- Das Direct Digital Control (DDC) Konzept aus der Vorlesung **muss** verwendet werden.

Analyse des Zeitverhaltens

- Bei „schnellem“ Drehen des Drehgebers erkennt Ihr Programm nicht alle Phasenwechsel und meldet in der Regel einen Fehler. In diesem Fall ist die Ausführung der Software zwischen dem Auslesen aufeinanderfolgender Phasen langsamer als der Phasenwechsel selbst.
- Aufgrund des verwendeten Bussystems ist die Kommunikation zwischen LCD-Display und CPU und somit die Ausgabe auf dem Display sehr langsam. Der Zeitbedarf der Ausgabe auf dem LCD-Display muss ausgemessen und dokumentiert werden. Vergleichen und dokumentieren Sie diesen mit der Ausführungszeit eines Super-Loop Durchlaufs.

- Führen Sie im Labor die Zeitmessungen mit dem Oszilloskop durch. Zur Messung schließen Sie einen freien GPIO Pin des ITS-Boards an einen Kanal des Oszilloskops an (Anschluss der Masse nicht vergessen). Setzen Sie dann vor der Ausgabe auf dem Display den GPIO Pin auf high. Ist die Ausgabe abgeschlossen, treiben Sie den Pin wieder mit low. Die mit dem Oszilloskop gemessene Impulsbreite entspricht (bis auf sehr kleine Abweichungen) der Ausführungszeit auf dem Display.

Zeitverhalten-bedingte Anforderungen

- Aufgrund der langsamen Displayausgabe werden in der Super-Loop nur Winkel und Winkelgeschwindigkeit ausgegeben – sofern sie sich geändert haben. Bildschirmtexte, die sich nicht ändern, werden nur einmalig in der Startphase ausgegeben und nicht überschrieben.
- Es ist sinnvoll, dass die Display-Ausgabe an die Anforderungen des Anwenders angepasst wird. Die Ausgabe sollte alle 250ms bis 500ms aktualisiert werden, wenn Winkel oder Winkelgeschwindigkeit sich geändert haben.
- Somit müssen Sie auch nur alle 250ms bis 500ms Winkel und Winkelgeschwindigkeit auf Basis der Anzahl der Phasenwechsel berechnen.
- Eine einfache Berechnung der Winkelgeschwindigkeit zählt in einem vorgegebenen Zeitfenster die Anzahl der Phasenwechsel und berechnet aus diesen Daten die Winkelgeschwindigkeit. Somit sollten die Zeitstempel, die Anfang und Ende des Zeitfensters festlegen, möglichst direkt hinter dem Zeitpunkt eines Phasenwechsels liegen.
- Da Winkel und Winkelgeschwindigkeit alle 250ms bis 500ms ausgegeben werden müssen, liegt die Breite des Zeitfensters zur Berechnung der Winkelgeschwindigkeit auch zwischen 250ms und 500ms.
- Nachdem 250ms des Zeitfensters vergangen sind, sollte Ihr Programm in jedem Super-Loop Durchlaufen schauen, ob ein Phasenwechsel vorliegt und dann den Zeitstempel für das Ende des Zeitfensters nehmen. Nachdem 500ms vergangen sind, wird das Zeitfenster geschlossen und die Berechnung von Winkel und Winkelgeschwindigkeit auf jeden Fall ausgeführt.

Abnahme

- Das Konzept muss vorliegen.
- Kommentierter Quellcode, der dem C-Coding Style entspricht, muss vorliegen.
- Überlegen Sie sich sinnvolle Testfälle. Eine Liste der von Ihnen durchgeführten Tests - zusammen mit den Testergebnissen - muss vorliegen.
- Die oben geforderte Analyse des Zeitverhaltens muss vorliegen.
- Sie müssen Ihr Programm erklären können.

Hinweise

- In dieser Aufgabe ist bei der Erstellung des Modulkonzepts, seiner Schnittstellen und seiner Funktionen das Design Prinzip „Separation of Concerns“ sehr wichtig: Damit Sie Überblick und Kontrolle über Ihr Programm behalten, ist es unbedingt erforderlich den Programmieraufwand anhand des Modulkonzepts im Team aufzuteilen.
- Für die Winkelberechnung benötigen Sie die Anzahl der Schlitze der Scheibe des Drehgebers. Der Drehgeber hat 300 Schlitze – also 1200 Phasenwechsel pro Umdrehung.

- Der Zugriff auf I/O erfolgt über Bit Manipulationen. Gehen Sie hier nach einem einheitlichen Verfahren vor (aus der Vorlesung).
- Sie müssen den Zustand des Drehgebers kodieren. Der Standardweg ist eine Finite State Machine (FSM). Wenn Sie FSMs schon kennen, wenden Sie diese gerne an. Ansonsten kann man den Zustand des Drehgebers auch ohne FSM gut umsetzen, indem Fehlerfall, Drehrichtung (rechts, links, nicht bekannt) und aktuelle Phase in statischen Variablen des entsprechenden Moduls gespeichert werden. Bei beiden Vorgehensweisen speichert eine Variable die Anzahl der Phasenwechsel, wobei die Drehrichtung beachtet wird.
- Der vorgegebene Timer (`timer.h`) ist so eingestellt, dass ein 32 Bit Register 90 mal in einer Mikrosekunde um 1 inkrementiert wird. Hat das Register den Wert `0xFFFFFFFF` erreicht, dann findet ein Überlauf statt und der nächste Registerwert ist `0x00000000`. Die Differenz zweier Werte, die aus dem Timer Register gelesen wurden, entsprechen der Zeitspanne zwischen den beiden lesenden Zugriffen auf das Register.